SECURITY

Security Policy

Supported versions

`marrow` is `0.1.0` (public API frozen per `STABILITY.md`). Only `main` receives security updates. There are no LTS branches, no backport policy, and no ABI-stability commitment across versions.

A full STRIDE analysis with severities and trust boundaries lives in `THREAT-MODEL.md`. This file is the *policy* (how to report, what's in scope); the threat model is the *analysis*.

Reporting a vulnerability

Do not open a public GitHub issue for security vulnerabilities.

If you find a security issue, please email the maintainers privately:

- Use GitHub's private security advisory feature
- Or open an issue titled "SECURITY — please contact me privately" with no details, and we will reach out

Please include:

1. A description of the issue and the affected component
2. Steps to reproduce, or a proof-of-concept
3. The version (`pip show marrow`) or commit SHA
4. Your environment (OS, Python version, threading model)
5. Any mitigations you've already identified

Response timeline

We aim to:

- Acknowledge receipt within **3 business days**
- Provide an initial assessment within **7 business days**
- Coordinate a fix with you before public disclosure

This is a best-effort commitment from volunteer maintainers, not an SLA.

Threat model — what is in and out of scope

`marrow` is a library, not a service. We treat the following as in scope:

- Memory corruption in the C++ core (use-after-free, buffer overrun, race conditions producing UB)
- GIL safety violations in Pybind11 bindings
- Code execution via crafted tool arguments or messages
- Denial of service via the public Python API with reasonable inputs
- Secrets leakage from provider implementations (e.g. API keys in error messages)

Out of scope:

- Vulnerabilities in optional dependencies (`openai`, `anthropic`, `httpx`) — please report those upstream
- Misuse: registering an obviously dangerous tool body (`subprocess.run`) and then complaining when it runs
- Side-channel attacks against the LLM provider itself (prompt injection, jailbreaking) — these are LLM concerns, not library concerns
- Insecure defaults in user code that imports `marrow`

Known security-relevant limitations

These are constraints to design around, not vulnerabilities in themselves. The list reflects the current state of `main` (see `THREAT-MODEL.md` for severities and IDs):

1. **Tool arguments are not validated against the registered schema.** The schema is informational; tool bodies must validate their own inputs. Argument and result **size caps are enforced** by `ToolBox` (1 MiB / 4 MiB), but not by the raw `ToolRegistry.register` path.
2. **Message size cap is enforced (4 MiB).** Enforced both in `Message.make` and on direct `.content` assignment. Oversized content is rejected, not truncated. (T2)
3. **Per-call timeouts and cancellation are enforced** at the SDK boundary: `step`/`stream` honor a `CancelToken`, and the OpenAI/Anthropic/Ollama providers pass `timeout_ms` through as a wall-clock timeout and re-check cancellation between stream chunks. A custom `Provider` you write should call `marrow.sdk.raise_if_cancelled(req)` at its yield points. (T1)
4. **`AgentRouter` inboxes are unbounded *by default*.** Set `router.set_inbox_limit(id, max, policy)` for backpressure. ARI-Embedded deployments **must** configure a finite bound. (T4)
5. **Tool error redaction is best-effort.** The default redactor scrubs common secret shapes and paths; for untrusted/multi-tenant use, construct `ToolBox(strict=True)` to return the exception *type only*. (T3)
6. **No per-agent tool ACL.** Any agent in a `Runtime` can invoke any registered tool; isolate trust domains with separate `Runtime`s. (T7)
7. **Provider `base_url` is trusted.** Never derive it from model output or untrusted input — a hostile URL exfiltrates prompts and API keys. (T8)
8. **At-rest conversation content is plaintext.** Use full-disk encryption or an encrypting `StateStore`; treat a shared DB file as a trust boundary. (T5)

Production users should still layer their own validation and least-privilege tool design on top.

Public disclosure

Once a fix is released, we will:

- Publish a GitHub Security Advisory describing the issue, affected

versions, and fix
- Credit the reporter (with your permission)
- Coordinate any CVE assignment if appropriate